

lab03

March 25, 2026

1 FFT lab: reproducing analytical FT with Python

Use the FFT as a **numerical approximation** of the continuous FT.

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-i2\pi ft} dt$$

and its inverse

$$x(t) = \int_{-\infty}^{\infty} X(f) e^{i2\pi ft} df$$

Main recipe:

$$t_n = nT, \quad f_s = \frac{1}{T}, \quad X(f_k) \approx T \text{FFT}\{x[n]\}$$

with corresponding frequencies

$$f_k = \text{fftfreq}(N, T).$$

```
[1]: import numpy as np
import matplotlib.pyplot as plt
print(np.fft.fftfreq(2))
print(np.fft.fftfreq(4))
print(np.fft.fftfreq(8))
print(np.fft.fftfreq(8, 0.1))
```

```
[ 0. -0.5]
[ 0.   0.25 -0.5 -0.25]
[ 0.   0.125 0.25 0.375 -0.5 -0.375 -0.25 -0.125]
[ 0.   1.25 2.5 3.75 -5. -3.75 -2.5 -1.25]
```

BEWARE! The frequency axis is not centered, i.e.,

instead of the structure: negative frequencies, 0, positive frequencies

we have the structure: 0, positive frequencies, negative frequencies

More precisely, the first $N/2$ elements ($[N/2:]$) of the FT correspond to amplitudes at zero and positive frequencies while the rest ($[N/2:]$) to negative frequencies.

```
[2]: np.fft.fftshift(np.fft.fftfreq(8, 0.1)) # center the frequencies
```

```
[2]: array([-5. , -3.75, -2.5 , -1.25,  0. ,  1.25,  2.5 ,  3.75])
```

Practical rules:

1. choose f_s high enough,
2. choose N (window size) large enough,
3. choose N to be integer power of two, e.g., 2^8
4. build the frequency axis explicitly,
5. scale by T ,
6. use `fftshift` when comparing with centered theoretical plots,
7. use long windows or edge decay for non-periodic signals,
8. zero-pad only for smoother display.

1.1 1. Minimal template

Use this every time:

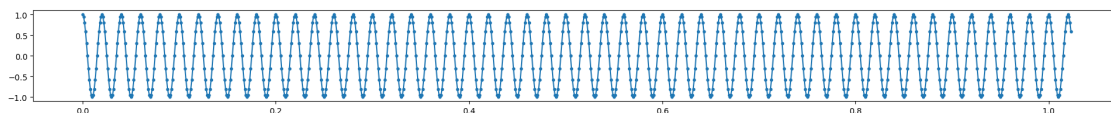
$$X(f_k) \approx T_s \sum_{n=0}^{N-1} x(t_n) e^{-i2\pi f_k t_n}.$$

So in code we use `Ts*np.fft.fft(x)`.

```
[3]: # Sampling grid
fs = 1000.0
T = 1 / fs
N = 1024
t = np.arange(N) * T

# Example signal
f0 = 50.0
x = np.cos(2 * np.pi * f0 * t)
plt.figure(figsize=(24, 2)) # wide and short
plt.plot(t, x, '.-')
```

```
[3]: [<matplotlib.lines.Line2D at 0x79e7f6738650>]
```



```
[4]: # FFT scaled to approximate the continuous FT
X = T * np.fft.fft(x)
f = np.fft.fftfreq(N, T)

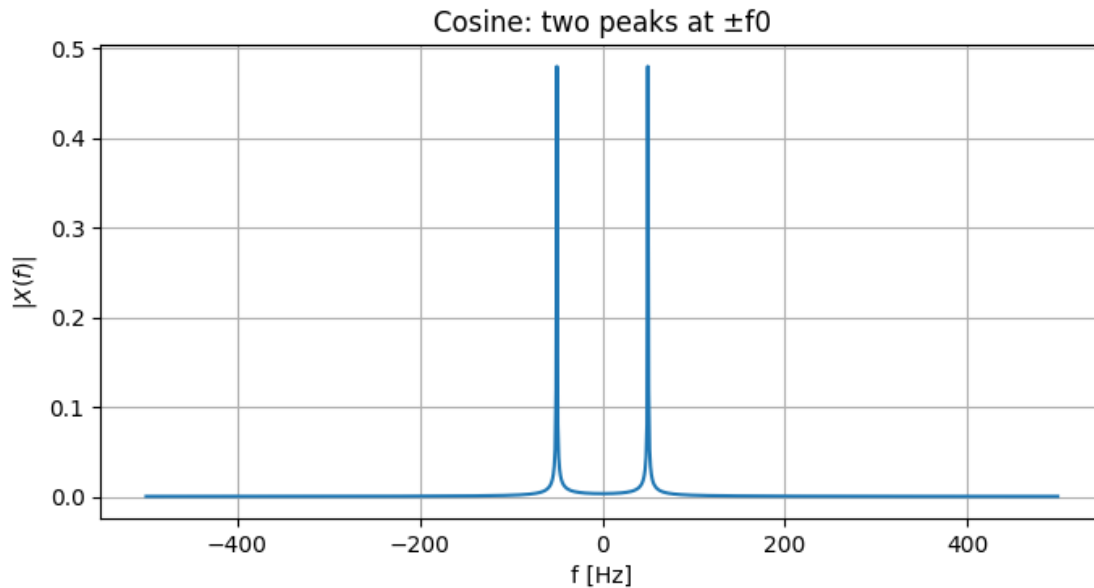
# Centered version for comparison with theoretical plots
Xc = np.fft.fftshift(X)
```

```

fc = np.fft.fftshift(f)

plt.figure(figsize=(8, 3.8))
plt.plot(fc, np.abs(Xc))
plt.xlabel("f [Hz]")
plt.ylabel(r"$|X(f)|$")
plt.title("Cosine: two peaks at  $\pm f_0$ ")
plt.grid(True)
plt.show()

```



1.2 2. Frequency grid and resolution

The FFT returns values only on discrete frequencies:

$$\Delta f = \frac{f_s}{N}.$$

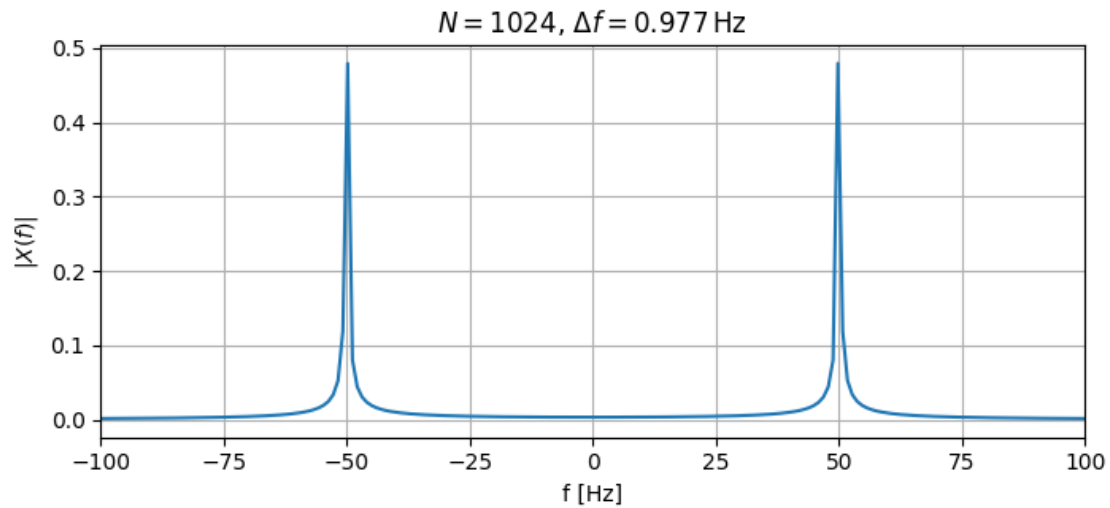
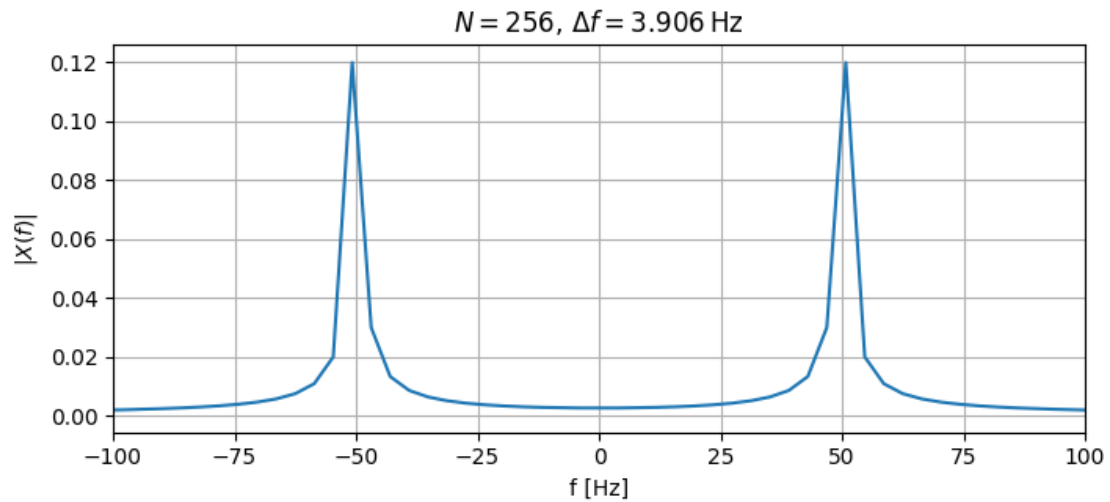
Larger (N) means denser frequency samples.

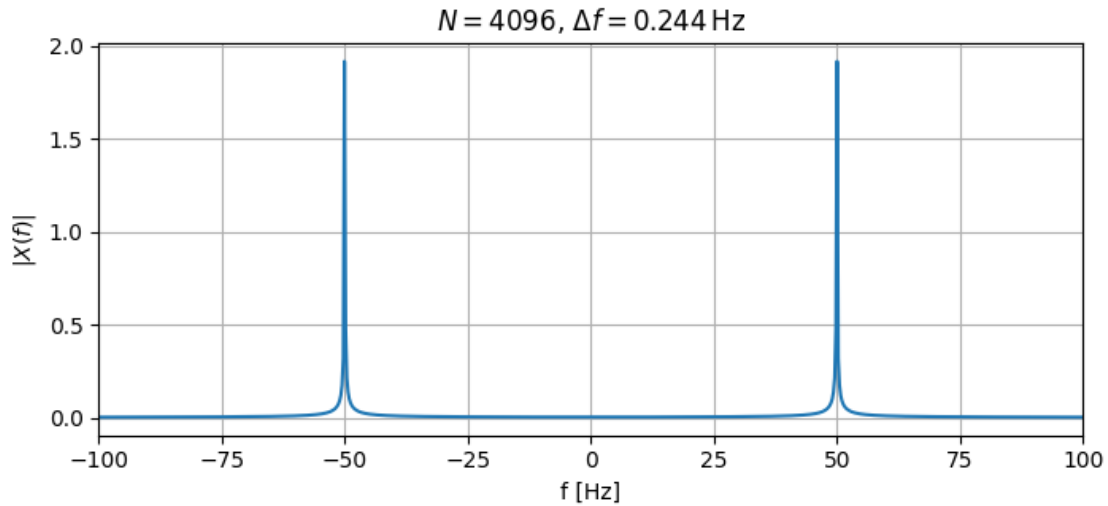
```

[5]: for N in [256, 1024, 4096]:
    t = np.arange(N) / fs
    x = np.cos(2 * np.pi * f0 * t)
    X = T * np.fft.fft(x)
    f = np.fft.fftfreq(N, T)
    Xc = np.fft.fftshift(X)
    fc = np.fft.fftshift(f)

```

```
plt.figure(figsize=(8, 3.2))
plt.plot(fc, np.abs(Xc))
plt.xlim(-100, 100)
plt.xlabel("f [Hz]")
plt.ylabel(r"$|X(f)|$")
plt.title(fr"$N={N}$, $\Delta f = {fs/N:.3f} \backslash, \mathrm{{Hz}}$")
plt.grid(True)
plt.show()
```





1.3 3. Window fit: good vs bad case

To make the FFT resemble the expected transform cleanly, it helps if the sampled signal fits the window well.

Example:

- good fit: integer number of periods in the window,
- bad fit: non-integer number of periods.

This is not a theory point; it is a **finite-window implementation issue**.

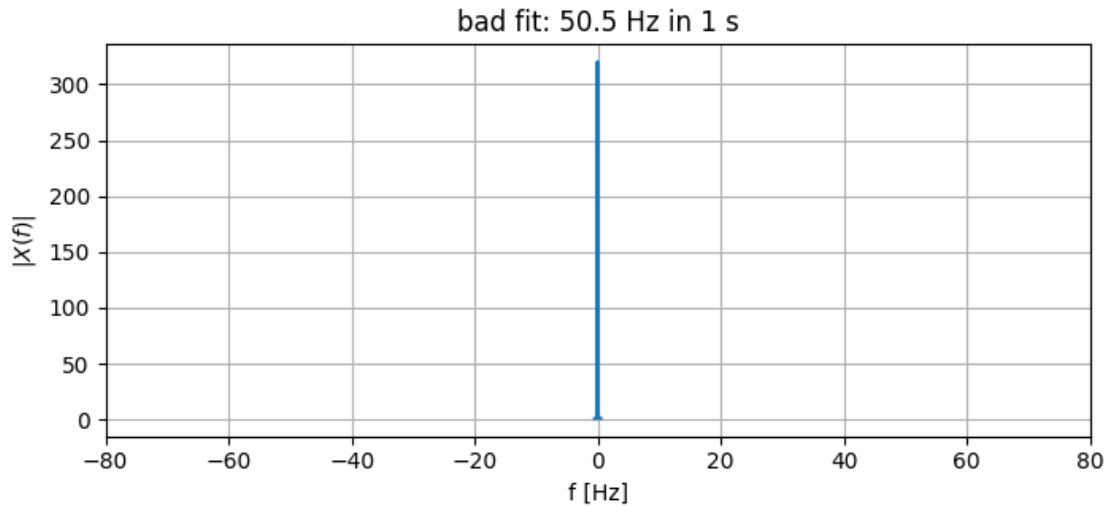
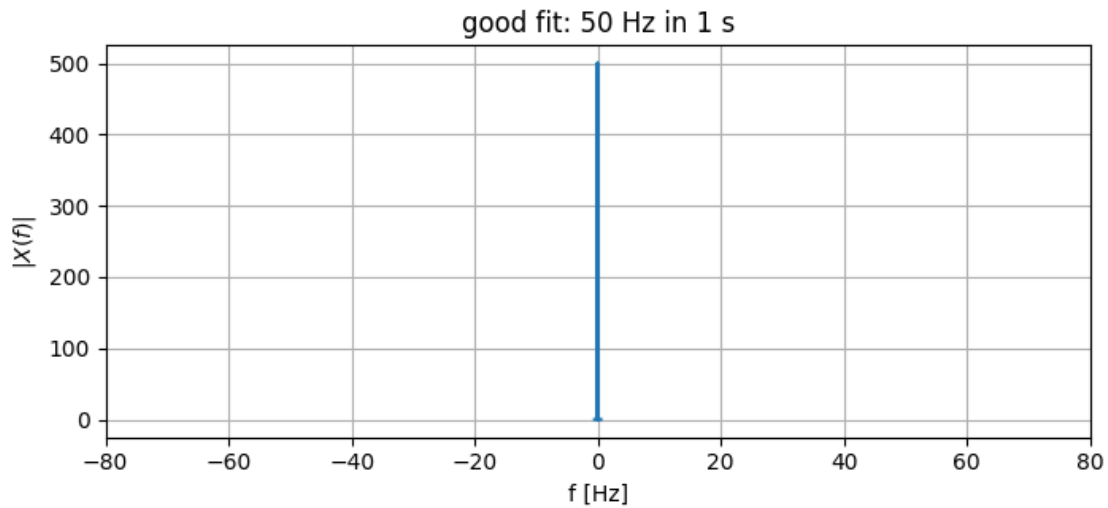
```
[7]: T = 1.0
N = int(fs * T)
t = np.arange(N) / fs

signals = [
    (50.0, "good fit: 50 Hz in 1 s"),
    (50.5, "bad fit: 50.5 Hz in 1 s"),
]

for f0, title in signals:
    x = np.cos(2 * np.pi * f0 * t)
    X = T * np.fft.fft(x)
    f = np.fft.fftfreq(N, T)
    Xc = np.fft.fftshift(X)
    fc = np.fft.fftshift(f)

    plt.figure(figsize=(8, 3.2))
    plt.plot(fc, np.abs(Xc))
    plt.xlim(-80, 80)
```

```
plt.xlabel("f [Hz]")
plt.ylabel(r"$|X(f)|$")
plt.title(title)
plt.grid(True)
plt.show()
```



1.4 4. Non-periodic functions: use a long enough window

Analytical transforms often assume infinite support, but the FFT only sees a finite interval.

Recipe: choose the window so that the function is already small near both ends.

Example:

$$x(t) = e^{-at^2}.$$

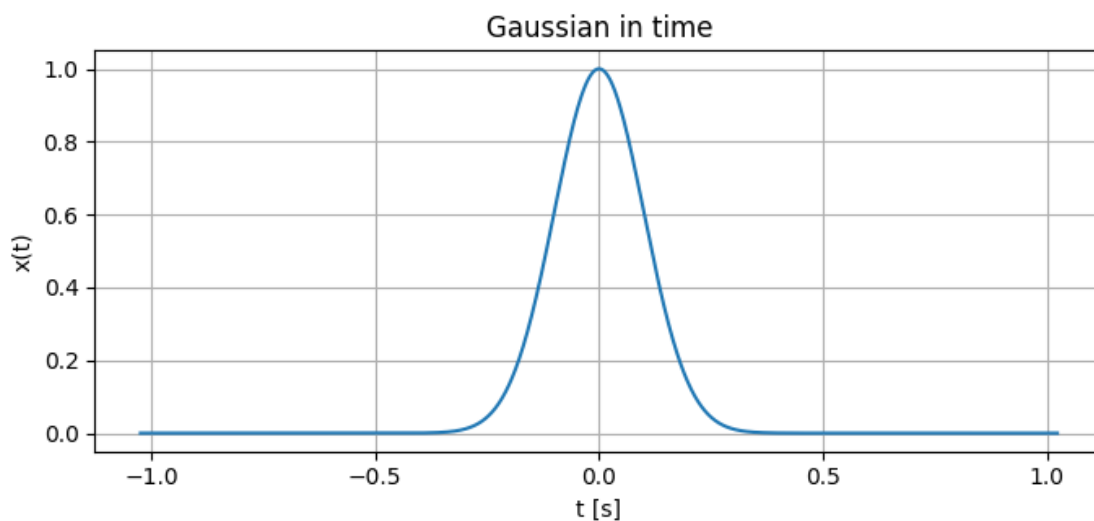
```
[14]: fs = 2000.0
T = 1 / fs
N = 4096
t = (np.arange(N) - N//2) * T  # centered time grid

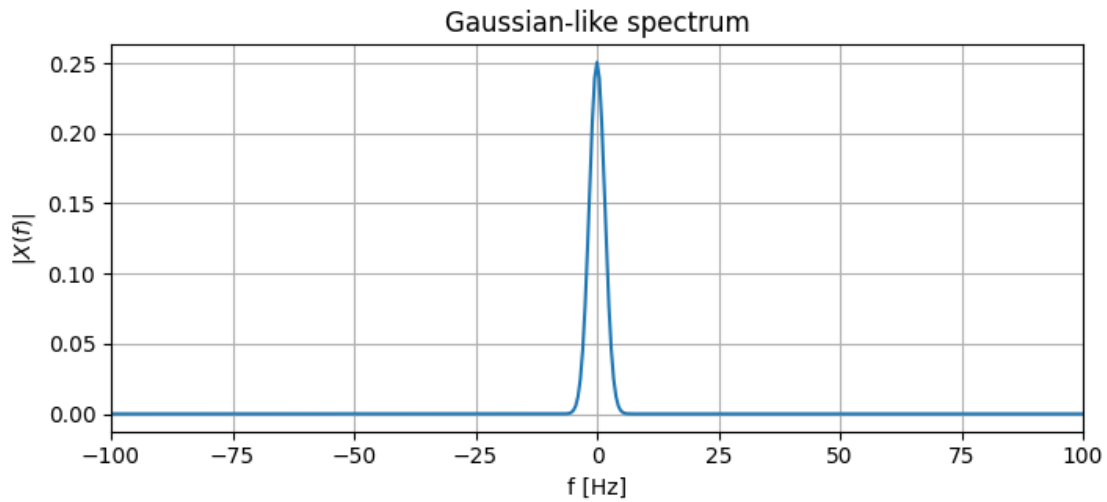
a = 50.0
x = np.exp(-a * t**2)

X = T * np.fft.fft(x)
f = np.fft.fftfreq(N, T)
Xc = np.fft.fftshift(X)
fc = np.fft.fftshift(f)

plt.figure(figsize=(8, 3.2))
plt.plot(t, x)
plt.xlabel("t [s]")
plt.ylabel("x(t)")
plt.title("Gaussian in time")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 3.2))
plt.plot(fc, np.abs(Xc))
plt.xlim(-100, 100)
plt.xlabel("f [Hz]")
plt.ylabel(r"$|X(f)|$")
plt.title("Gaussian-like spectrum")
plt.grid(True)
plt.show()
```





1.5 Example: Rectangular pulse $\rightarrow$ sinc-shaped spectrum

A compact numerical demonstration of a standard transform pair.

$$x(t) = \text{rect}\left(\frac{t}{T_0}\right).$$

Again: use a centered grid and scale by T .

```
[31]: fs = 4000.0
      T = 1 / fs
      N = 4096
      t = (np.arange(N) - N//2) * T

      T0 = 0.04
      x = (np.abs(t) <= T0/2).astype(float)

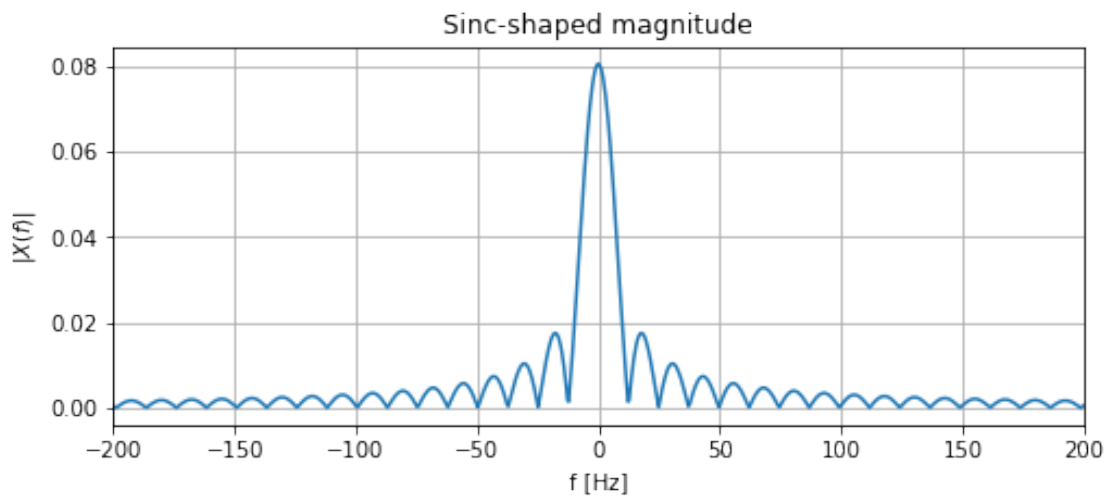
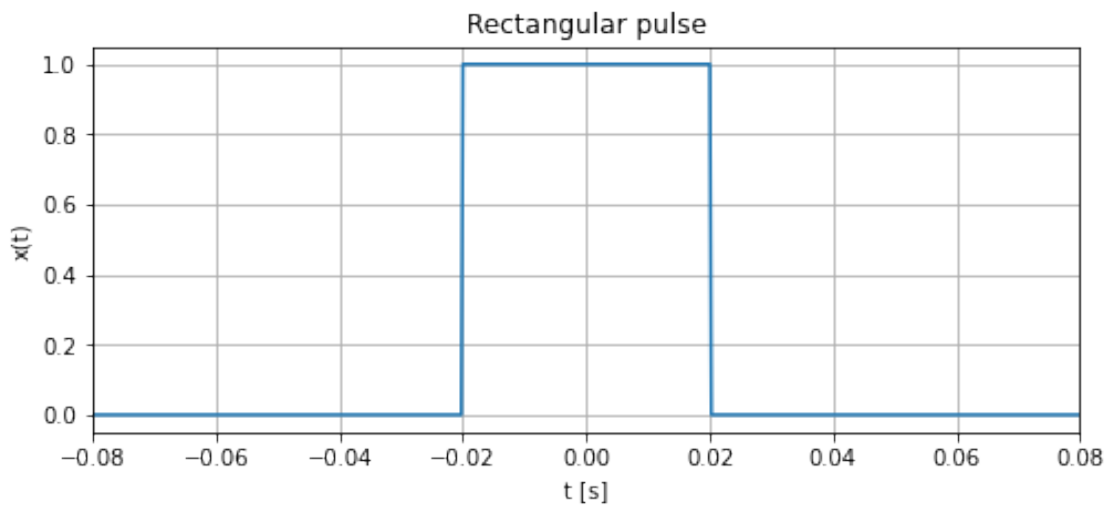
      X = Ts * np.fft.fft(x)
      f = np.fft.fftfreq(N, Ts)
      Xc = np.fft.fftshift(X)
      fc = np.fft.fftshift(f)

      plt.figure(figsize=(8, 3.2))
      plt.plot(t, x)
      plt.xlim(-0.08, 0.08)
      plt.xlabel("t [s]")
      plt.ylabel("x(t)")
```



```
plt.title("Rectangular pulse")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 3.2))
plt.plot(fc, np.abs(Xc))
plt.xlim(-200, 200)
plt.xlabel("f [Hz]")
plt.ylabel(r"$|X(f)|$")
plt.title("Sinc-shaped magnitude")
plt.grid(True)
plt.show()
```



[]: